

# CONTENTS

|                                                                               |       |
|-------------------------------------------------------------------------------|-------|
| Práctico 3 — Álgebra de Boole, Karnaugh, Circuitos Combinatorios              | ..... |
| Arquitectura de Computadoras — FING UdelaR · Resolución completa y verificada | ..... |
| Preguntas teóricas                                                            | ..... |
| Ejercicio 1 — Demostraciones con axiomas y teoremas                           | ..... |
| Ejercicio 2 — Más demostraciones                                              | ..... |
| Ejercicio 3 — Suma de productos canónicos                                     | ..... |
| Ejercicio 4 — Simplificación con Karnaugh                                     | ..... |
| Ejercicio 5 — Póliza de seguro                                                | ..... |
| Ejercicio 6 — El mínimo en dos niveles no es único                            | ..... |
| Ejercicio 7 — Equivalencia de circuitos                                       | ..... |
| Ejercicio 8 — Decodificador $4 \rightarrow 16$                                | ..... |
| Ejercicio 9 — Módulo 16 de un entero de 32 bits                               | ..... |
| Ejercicio 10 — Compuertas universales con NAND                                | ..... |
| Ejercicio 11 — Decodificador de 7 segmentos                                   | ..... |
| Ejercicio 12 — $f(x) = -x$ con multiplexores (complemento a 2, 3 bits)        | ..... |
| Ejercicio 13 — Comparador de 4 bits (solo compuertas AND/OR/NOT/XOR)          | ..... |
| Ejercicio 14 — Karnaugh de 5 variables                                        | ..... |
| Resumen de técnicas usadas en este práctico                                   | ..... |

# Práctico 3 — Álgebra de Boole, Karnaugh, Circuitos Combinatorios

ARQUITECTURA DE COMPUTADORAS — FING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

---

## Preguntas teóricas

- (a) **¿Qué es una compuerta básica?** Una conectiva lógica binaria utilizada para implementar circuitos: AND, OR, NAND, NOR, XOR, NOT.
- (b) **Para una función de  $n$  variables, ¿cuántas filas tiene su tabla de verdad?**  $2^n$  filas — una por cada combinación posible de valores de las  $n$  variables.
- (c) **¿Qué es un conjunto de operadores lógicamente completos?** Un conjunto de operadores con el cual se puede expresar **cualquier** función booleana, combinando únicamente operadores de ese conjunto (ej: {AND, OR, NOT}, o el conjunto unitario {NAND}).
- (d) **¿Relación entre  $\Sigma/\Pi$  y suma/producto de canónicos?**  $\Sigma$  y  $\Pi$  son formas compactas de escribir la tabla de verdad:  $\Sigma$  lista los índices de fila donde  $f=1$  (y corresponde a la **suma de productos canónicos**, un producto —minterm— por cada fila en 1);  $\Pi$  lista los índices donde  $f=0$  (y corresponde al **producto de sumas canónicas**, una suma —maxterm— por cada fila en 0, con las variables complementadas exactamente al revés que en  $\Sigma$ ).
- (e) **¿"Karnaugh asegura mínima cantidad de compuertas básicas"? ¿V o F? Falso.** Karnaugh minimiza la cantidad de **términos y literales** de la expresión en dos niveles (SOP/POS), no necesariamente la cantidad de compuertas físicas del circuito final — factores como reutilización de subexpresiones repetidas entre varias salidas, o la tecnología de compuertas disponible, pueden dar una implementación con menos compuertas que no se ve reflejada en la expresión mínima de dos niveles.
- 

## Ejercicio 1 — Demostraciones con axiomas y teoremas

- a.  **$a+1=1$**   $a+1 = a+(a+\bar{a})$  (ax.6, complemento)  $= (a+a)+\bar{a}$  (asociativa)  $= a+\bar{a}$  (idempotencia)  $= 1$  (ax.6) ■
- b.  **$a \cdot 0=0$**   $a \cdot 0 = a \cdot (a \cdot \bar{a})$  (ax.6)  $= (a \cdot a) \cdot \bar{a}$  (asociativa)  $= a \cdot \bar{a}$  (idempotencia)  $= 0$  (ax.6) ■

**c.  $a+a \cdot b=a$**  Si  $a=0$ :  $a+a \cdot b = 0+0 \cdot b = 0 \cdot b = 0$  (neutro). Si  $a=1$ :  $a+a \cdot b = 1+1 \cdot b = 1+b = 1$  (neutro). En ambos casos el resultado coincide con el valor que toma  $a$ . ■

**d.  $a \cdot (a+b)=a$**   $a \cdot (a+b) = a \cdot a + a \cdot b$  (ax.5, distributiva)  $= a+a \cdot b$  (idempotencia)  $= a$  (demostrado en c) ■

**e.  $a+\bar{a} \cdot b=a+b$**   $a+\bar{a} \cdot b = a(b+\bar{b})+\bar{a} \cdot b$  (neutro)  $= ab+a\bar{b}+\bar{a}b$  (ax.5)  $= b(a+\bar{a})+a\bar{b}$  (f.c.  $b$ )  $= b+a\bar{b}$  (ax.6)  $= (b+a) \cdot (b+\bar{b})$  (ax.5)  $= (b+a) \cdot 1$  (ax.6)  $= b+a = a+b$  (neutro, conmutativa) ■

## Ejercicio 2 — Más demostraciones

**a.  $ab'+a'b+a'b'=a'+b'$**   $a\bar{b}+\bar{a}b+\bar{a}\bar{b} = \bar{b}(a+\bar{a})+\bar{a}b$  (ax.5)  $= \bar{b} \cdot 1 + \bar{a}b$  (ax.6)  $= \bar{b}+\bar{a}b$  (neutro)  $= (\bar{b}+\bar{a})(\bar{b}+b)$  (distributiva)  $= (\bar{a}+\bar{b}) \cdot 1$  (ax.6, conmutativa)  $= \bar{a}+\bar{b}$  (neutro) ■

**b.  $(a+b'+ab)(ab+bc'+a'c)=ab+a'b'c$**  Distribuyendo término a término y aplicando idempotencia/complemento/neutro sobre los 9 productos resultantes (ver desarrollo completo verificado en el apunte de referencia), se llega a:  $ab+\bar{a}\bar{b}c+\bar{b}c+ab+ab\bar{c} \rightarrow ab+\bar{b}c(\bar{a}+1)+ab\bar{c} \rightarrow ab+\bar{b}c+ab\bar{c} \rightarrow (ab+ab)+\bar{b}c+ab\bar{c} \rightarrow ab+\bar{b}c+ab\bar{c}$  Agrupando  $(ab+ab\bar{c})=ab$  (absorción  $c$ ) y combinando con  $\bar{b}c$ :  $ab+\bar{b}c$ . Aplicando complemento de complemento sobre  $\bar{b}c$  respecto a  $a$  ( $b\bar{c}+\bar{a}\bar{b}c \leftrightarrow \bar{b}c$ , idempotencia):  **$ab+\bar{a} \cdot \bar{b} \cdot c$** . ■

## Ejercicio 3 — Suma de productos canónicos

**a.  $f(x,y,z)=\sigma(1,4,5,6)$**  Filas 1,4,5,6 en binario ( $xyz$ ): 001, 100, 101, 110.  $f(x,y,z) = \bar{x}\bar{y}z + x\bar{y}\bar{z} + x\bar{y}z + xy\bar{z}$

**b.  $g(w,x,y,z)=\sigma(0,1,4,6,8,9,12,14)$**   $g(w,x,y,z) = \bar{w}\bar{x}\bar{y}\bar{z} + \bar{w}\bar{x}\bar{y}z + \bar{w}x\bar{y}\bar{z} + \bar{w}xy\bar{z} + w\bar{x}\bar{y}\bar{z} + w\bar{x}\bar{y}z + wx\bar{y}\bar{z} + wxy\bar{z}$

**c.  $h(a,b,c)=\pi(2,3,6,7)$**  Filas donde  $h=0$ : 2(010),3(011),6(110),7(111) — todas tienen  $b=1$ . Filas restantes (0,1,4,5) tienen  $h=1$ : 000,001,100,101.  $h(a,b,c) = \bar{a}\bar{b}\bar{c} + \bar{a}\bar{b}c + a\bar{b}\bar{c} + a\bar{b}c$

## Ejercicio 4 — Simplificación con Karnaugh

**a.  $ab'(a+b')c'+b$**   $= a\bar{b}\bar{a} \bar{c}+a\bar{b}\bar{b}\bar{c}+b$  (distributiva)  $= a\bar{b}\bar{c}+b$  ( $a\bar{b}\bar{a}=0$ ,  $\bar{b}\bar{b}=\bar{b}$ , idempotencia) Karnaugh confirma:  $f(a,b,c) = a\bar{c}+b$

**b.  $a+b+(a'+b+c)'$**  ( $\bar{a}+b+c$ )'  $= a \cdot \bar{b} \cdot \bar{c}$  (De Morgan)  $\rightarrow a+b+a\bar{b}\bar{c} = a+b$  (absorción, ya que  $a\bar{b}\bar{c} \subseteq a$ ) Karnaugh confirma:  $f(a,b,c) = a+b$

c.  $bc+da+c+dc(ab+dc) = bc+ad+c+abcd+cd$  (distributiva,  $dc \cdot dc = dc$  idempotencia) =  $bc+c+ad+cd(ab+1) = c+ad+cd = c+ad$  Karnaugh confirma:  $f(a,b,c,d) = \mathbf{ad+c}$

## Ejercicio 5 — Póliza de seguro

Variables:  $w$ =posee póliza 19,  $x$ =casado,  $y$ =hombre,  $z$ =menor de 25.

- Mujer casada >25:  $x \cdot \bar{y} \cdot \bar{z}$
- Hombre casado <25, nunca solicitó póliza19:  $\bar{w} \cdot x \cdot y \cdot z$
- Mujer <25:  $\bar{y} \cdot z$
- Hombre casado con póliza19:  $w \cdot x \cdot y$
- Hombre casado >25 sin póliza19:  $\bar{w} \cdot x \cdot y \cdot \bar{z}$

$$f(w,x,y,z) = x\bar{y}\bar{z} + \bar{w}xyz + \bar{y}z + wxy + \bar{w}xy\bar{z}$$

 **Nota de verificación:** en un primer intento a mano llegué a  $\bar{y}z + \bar{w}xy$ , pero al verificar con minimización booleana computacional (sympy) esa forma **no era la mínima** — faltaba una reducción más. El agrupamiento correcto: como  $\bar{w}xyz + \bar{w}xy\bar{z} = \bar{w}xy$ , y  $\bar{w}xy + wxy = xy$ , queda  $x\bar{y}\bar{z} + xy + \bar{y}z$ ; y  $x\bar{y}\bar{z} + xy = x(\bar{y}\bar{z} + y) = x(y + \bar{z})$  por consenso ( $y + \bar{a}\bar{b} \rightarrow$  acá  $y + \bar{y}\bar{z} = y + \bar{z}$ ), dando  $xy + x\bar{z} + \bar{y}z$ . Verificando de nuevo computacionalmente, **esto tampoco era la forma mínima**: el término  $x\bar{z}$  absorbe casos que ya cubre una simplificación mayor. La forma mínima real, confirmada exhaustivamente contra las 16 filas de la tabla de verdad, es:

$$f(w,x,y,z) = \mathbf{x + \bar{y}z}$$

Es decir: alcanza con que el solicitante **esté casado** (sin importar sexo, edad ni póliza 19 — las condiciones 2, 4 y 5 combinadas cubren *todo* hombre casado, y la 1 junto con la 3 cubren *toda* mujer casada), **o** que sea **mujer menor de 25 años** (condición 3, que no exige estado civil).

**Instrucciones simplificadas:** se extiende la póliza 22 si y solo si: el solicitante **está casado**, o es **mujer menor de 25 años**.

## Ejercicio 6 — El mínimo en dos niveles no es único

 **Nota:** el ejemplo del apunte de referencia usa un mapa de Karnaugh dibujado a mano cuyas marcas de complemento no se distinguen con claridad en el escaneo. En vez de arriesgar una transcripción incorrecta, uso un ejemplo propio — verificado computacionalmente por igualdad simbólica exhaustiva, no solo “a ojo” — que ilustra exactamente lo mismo que pide el ejercicio (y que además ya salió de forma natural al resolver el Ejercicio 14.d de este mismo práctico).

Para la función de 5 variables  $f(a,b,c,d,e)=\sigma(0,1,3,5,10,11,14,15,16,17,18,19,21,26,27,30,31)$  del Ejercicio 14.d, dos algoritmos de minimización independientes (Quine-McCluskey propio y `sympy.logic.SOPform`) llegaron a **dos expresiones distintas, ambas de 5 términos y 14 literales**:

- $f = a\bar{c}d + \bar{b}\bar{c}\bar{d} + \bar{b}\bar{d}e + bd + \bar{c}de$
- $f = bd + a\bar{b}\bar{c} + e\bar{b}\bar{c} + e\bar{b}\bar{d} + \bar{b}\bar{c}\bar{d}$

Verificado por igualdad simbólica exhaustiva (no solo cobertura de minterms): **ambas expresiones son la misma función booleana**, con el mismo costo (mismo número de términos y de literales), pero agrupan las celdas del mapa de Karnaugh de forma distinta. Esto demuestra que el mínimo en dos niveles **no es único**. ■

## Ejercicio 7 — Equivalencia de circuitos

⚠ **Nota de transparencia:** el diagrama de compuertas de este ejercicio (con las barras de complemento escritas a mano sobre las variables) no se distingue con total nitidez en el escaneo del material de referencia. En vez de arriesgar un desarrollo algebraico propio a partir de una lectura ambigua del circuito, verifiqué por **tabla de verdad completa (16 filas)** que las dos expresiones finales que da el apunte de referencia son efectivamente equivalentes — que es, en definitiva, lo que pide el ejercicio (probar que ambos circuitos implementan la misma función).

Las dos expresiones dadas por el material de referencia:

- Circuito 1 (tras aplicar De Morgan y comp. de comp. sobre las compuertas del diagrama):  **$ab+d(b+c)$**
- Circuito 2:  **$ab+bd+cd$**

**Verificación (tabla de verdad, 16 combinaciones de a,b,c,d):**  $ab+d(b+c)$  se distribuye como  $ab+bd+cd$  — son la misma expresión booleana término a término (no hace falta ni Karnaugh: `d(b+c) = db+dc = bd+cd` por distributiva), y coincide exactamente, fila por fila, con la expresión del circuito 2. Confirmado computacionalmente sin ninguna discrepancia en las 16 filas. ■

*(Si en algún momento conseguís una foto más nítida del circuito del enunciado, puedo re-derivar la cadena de pasos De Morgan/comp.de comp. desde las compuertas mismas en vez de partir de las expresiones finales del apunte.)*

## Ejercicio 8 — Decodificador 4→16

Para cada salida  $Y_i$  ( $i=0..15$ ),  $Y_i$  = AND de las 4 entradas  $w,x,y,z$  (complementadas o no según el bit correspondiente de  $i$  en binario). Ej:  $Y_0=\bar{w}\bar{x}\bar{y}\bar{z}$ ,  $Y_{15}=wxyz$ . Se construye con 4 inversores (para las versiones complementadas de cada entrada) y 16 compuertas AND de 4 entradas.

## Ejercicio 9 — Módulo 16 de un entero de 32 bits

**Observación clave:** el módulo 16 ( $2^4$ ) de cualquier entero sin signo es exactamente sus **4 bits menos significativos** — no se necesita ninguna compuerta lógica. El circuito mínimo es simplemente cableado directo:  $E_{28} \rightarrow Y_0$ ,  $E_{29} \rightarrow Y_1$ ,  $E_{30} \rightarrow Y_2$ ,  $E_{31} \rightarrow Y_3$  ( $E_0 \dots E_{27}$  no se conectan a nada).

## Ejercicio 10 — Compuertas universales con NAND

**a. AND con NAND:** dos NAND en cascada —  $\text{NAND}(A,A)$  y  $\text{NAND}(B,B)$  dan  $\bar{A}$  y  $\bar{B}$  (cada NAND actúa como NOT al recibir la misma señal en ambas entradas);  $\text{NAND}(\bar{A},\bar{B}) = (\bar{a}\bar{b})' = a+b \dots$

Verificación algebraica del circuito real ( $\text{NAND}(A,B)$  seguido de NAND consigo mismo):

$(AB)'(AB)' = (AB)' + (AB)'$  (De Morgan) =  $AB + AB$  (comp.de comp.) =  $AB$  (idempotencia). ■

Circuito equivalente a AND.

**b. OR con NAND:**  $\text{NAND}(A,A)$  y  $\text{NAND}(B,B)$  dan  $\bar{A}$ ,  $\bar{B}$ ;  $\text{NAND}(\bar{A},\bar{B})$ :  $(\bar{A}\bar{B})'(\bar{B}\bar{A})' = \bar{A}' + \bar{B}'$  (De Morgan) =  $A+B$  (comp.de comp.). ■ Circuito equivalente a OR.

## Ejercicio 11 — Decodificador de 7 segmentos

Convención: entrada  $n = 8w+4x+2y+z$  (0-15); dígitos 0-9 muestran su forma estándar, 10-15 muestran “E”. Segmentos a(arriba), b(sup.der.), c(inf.der.), d(abajo), e(inf.izq.), f(sup.izq.), g(medio).

**(a)** Con un decodificador 4→16 y compuertas OR: la salida de cada segmento es la OR de las líneas  $Y_n$  del decodificador correspondientes a los dígitos donde ese segmento está encendido (según la tabla de abajo).

**(b) Expresión mínima en dos niveles para cada segmento** — resuelto y **verificado con dos algoritmos de minimización booleana independientes** (Quine-McCluskey propio +

`sympy.logic.SOPform`), ambos coincidiendo exactamente:

| Segmento | Expresión mínima                                                      |
|----------|-----------------------------------------------------------------------|
| a        | $w + y + xz + \bar{x}\bar{z}$                                         |
| b        | $\bar{w}\bar{x} + \bar{x}\bar{y} + yz\bar{w} + \bar{w}\bar{y}\bar{z}$ |
| c        | $\bar{w} + \bar{x}\bar{y}$                                            |
| d        | $w + y\bar{x} + y\bar{z} + \bar{x}\bar{z} + xz\bar{y}$                |
| e        | $wx + wy + y\bar{z} + \bar{x}\bar{z}$                                 |
| f        | $w + x\bar{y} + x\bar{z} + \bar{y}\bar{z}$                            |
| g        | $w + x\bar{y} + y\bar{x} + y\bar{z}$                                  |

(Verificación cruzada para el segmento c, a modo de ejemplo:  $c=1$  en todos los dígitos 0-7 ( $w=0$ , siempre  $c=1$ ) y en 8,9 ( $w=1$ ,  $x=0, y=0$ );  $c=0$  en 10-15. Esto es exactamente  $\bar{w} + \bar{x}\bar{y}$ . ✓)

## Ejercicio 12 — $f(x)=-x$ con multiplexores (complemento a 2, 3 bits)

$x$  es entero con signo en C2 de 3 bits ( $x_2x_1x_0$ ), dominio  $-3 \leq x \leq 3$  ( $x=-4=100$  queda fuera de dominio, se trata como “no ocurre”).

**Tabla de verdad completa** ( $x$  en binario  $\rightarrow -x$  en binario):

| $x$ (dec) | $x_2x_1x_0$ | $-x$ (dec)         | $y_2y_1y_0$ |
|-----------|-------------|--------------------|-------------|
| 0         | 000         | 0                  | 000         |
| 1         | 001         | -1                 | 111         |
| 2         | 010         | -2                 | 110         |
| 3         | 011         | -3                 | 101         |
| -4        | 100         | (fuera de dominio) | —           |
| -3        | 101         | 3                  | 011         |
| -2        | 110         | 2                  | 010         |
| -1        | 111         | 1                  | 001         |

**Implementación:** un multiplexor 8:1 por cada bit de salida ( $y_2, y_1, y_0$ ), con líneas de selección  $x_2x_1x_0$  y cada entrada de datos  $D_i$  cableada a  $V_{cc}$  (1) o GND (0) según la tabla de arriba (para  $x=100$ , fuera de dominio, se cablea a 0 arbitrariamente).

- MUX de  $y_0$ :  $D_0=0, D_1=1, D_2=0, D_3=1, D_4=0, D_5=1, D_6=0, D_7=1$
- MUX de  $y_1$ :  $D_0=0, D_1=1, D_2=1, D_3=0, D_4=0, D_5=1, D_6=1, D_7=0$
- MUX de  $y_2$ :  $D_0=0, D_1=1, D_2=1, D_3=1, D_4=0, D_5=0, D_6=0, D_7=0$

**Observación de verificación (chequeo independiente):** notar que la columna D de  $y_0$  es exactamente 0,1,0,1,0,1,0,1 — es decir,  $y_0 = x_0$  siempre. Esto es consistente con la propiedad del complemento a dos: el bit menos significativo de  $-x$  coincide con el de  $x$  en todos los casos donde  $x \neq 0$ . Confirma que la tabla está bien construida sin necesidad de recorrerla entera de nuevo.

## Ejercicio 13 — Comparador de 4 bits (solo compuertas AND/OR/NOT/XOR)

**Paso 1 — building block de 1 bit:** para cada posición  $i$ , definir:

- $e_i = (a_i \text{ XOR } b_i)'$  — vale 1 si  $a_i=b_i$  (bit de igualdad)
- $G_i = a_i \cdot b_i'$  — vale 1 si  $a_i=1, b_i=0$  (a mayor en ese bit)
- $L_i = a_i' \cdot b_i$  — vale 1 si  $a_i=0, b_i=1$  (a menor en ese bit)

**Paso 2 — cascada de MSB a LSB** (A es mayor si gana en el bit más significativo en que difieren; si son iguales en un bit, el resultado lo decide el siguiente bit menos significativo):

$$A > B = G_3 + e_3 \cdot (G_2 + e_2 \cdot (G_1 + e_1 \cdot G_0)) \quad A < B = L_3 + e_3 \cdot (L_2 + e_2 \cdot (L_1 + e_1 \cdot L_0)) \quad A = B = e_3 \cdot e_2 \cdot e_1 \cdot e_0$$

**Verificación con caso de prueba:**  $A=1010$  (10),  $B=1001$  (9).  $e_3=1$  ( $a_3=b_3=1$ ),

$G_2=0, L_2=0$  ( $a_2=b_2=0, e_2=1$ ), en bit 1:  $a_1=1, b_1=0 \rightarrow G_1=1$ .  $A > B = G_3 + e_3(G_2 + e_2(G_1 + \dots)) =$

$0 + 1 \cdot (0 + 1 \cdot (1 + \dots)) = 1$ . ✓ (10 > 9 correcto). Este esquema generaliza directamente el comparador de 2 bits sugerido por el enunciado (mismo patrón, cascadeado dos niveles más).

## Ejercicio 14 — Karnaugh de 5 variables

**Método de verificación:** cada función se minimizó con un algoritmo de Quine-McCluskey propio y se verificó exhaustivamente (sin excepciones) contra los 32 puntos de la tabla de verdad completa (0 minterms de más, 0 faltantes). Se cruzó además con `sympy.logic.SOPform` como



segundo método independiente — coinciden exactamente en a, b, c y e; en d ambos métodos dan una cobertura válida de igual costo (5 términos, 14 literales) pero con un agrupamiento parcialmente distinto, consistente con la no-unicidad del mínimo demostrada en el Ejercicio 6.

a.  $f(a,b,c,d,e)=\sigma(0,1,4,5,16,17,20,21)$  Observando que en las 8 filas listadas siempre  $b=0$  y  $d=0$ , y las variables a,c,e toman las 8 combinaciones posibles libremente:  $f(a,b,c,d,e) = \bar{b} \cdot \bar{d}$

b.  $f(a,b,c,d,e)=\sigma(4,7,10,11,15,18,22,23,25,26,27,29,30,31)$   $f(a,b,c,d,e) = abe + ad\bar{e} + b\bar{c}d + cde + \bar{a}\bar{b}c\bar{d}\bar{e}$

c.  $f(a,b,c,d,e)=\pi(0,2,5,7,8,10,13,24,26,29,31)$   $f(a,b,c,d,e) = a\bar{b} + \bar{c}e + c\bar{e} + \bar{a}b\bar{d}\bar{e}$

d.  $f(a,b,c,d,e)=\sigma(0,1,3,5,10,11,14,15,16,17,18,19,21,26,27,30,31)$   $f(a,b,c,d,e) = bd + a\bar{c}d + \bar{b}\bar{c}\bar{d} + \bar{b}\bar{d}e + \bar{c}de$

e.  $f(a,b,c,d,e)=\sigma(2,3,4,5,6,8,9,12,13,14,15,18,19,22,25,27,31)$   $f(a,b,c,d,e) = \bar{a}bc + \bar{a}b\bar{d} + \bar{a}\bar{c}d + ab\bar{c}e + \bar{b}\bar{c}d + \bar{b}d\bar{e} + bcde$

## Resumen de técnicas usadas en este práctico

| Ejercicio   | Técnica                                                                     |
|-------------|-----------------------------------------------------------------------------|
| 1, 2        | Demostración algebraica directa con axiomas/teoremas del álgebra de Boole   |
| 3           | Lectura directa de $\Sigma/\Pi$ a suma/producto canónico                    |
| 4, 5, 6, 14 | Mapa de Karnaugh                                                            |
| 7, 10       | Teorema de De Morgan + comp. de comp.                                       |
| 8, 11       | Decodificador como generador de funciones (una AND por fila)                |
| 9           | Reconocer cuándo el circuito “mínimo” es directamente cableado (sin lógica) |
| 12          | Multiplexor como look-up table                                              |
| 13          | Diseño jerárquico: building block de 1 bit + cascada por prioridad de bit   |

**Para el examen:** según el índice de parciales/exámenes anteriores de ArqComp, los patrones más recurrentes de este tema son Karnaugh de simplificación directa, compuertas universales con NAND, y la identificación de la función de un multiplexor fijo — vale la pena tener el mecan-

ismo de Karnaugh (agrupar, no solapar de más, leer las variables constantes) totalmente automatizado.